

qmeas

version 2.1

user manual

Instrument control for cryogenic magneto-transport measurements

License and liability • installation • connecting instruments
the task grid • nesting and linking • while-loops • scripts • running

Read the license and liability sections before relying on qmeas for any measurement.

The rest is short — skim §3–§4 once, then jump to whatever you need.

*This manual was generated entirely by an AI system and may contain errors.
Verify any critical detail against the software itself and your instruments' own documentation.*

qmeas and this manual were developed with Anthropic's Claude.

Contents

1	License	2
2	Liability and safety disclaimer	2
3	Installation	3
3.1	Step 1 — Python	3
3.2	Step 2 — Python packages	3
3.3	Step 3 — Run qmeas	4
4	Optional — the AI Assistant (Ollama)	4
4.1	Step 1 — Install Ollama	4
4.2	Step 2 — Pull a model	4
4.3	Step 3 — Hardware and model size	5
4.4	Step 4 — Use it	5
5	The five panels	5
6	Connecting a device	6
6.1	Scan first — but don't trust an empty result	6
6.2	Adding a socket (LAN) device by IP	6
6.3	HTTP / JSON-RPC devices	6
6.4	Termination characters matter	7
6.5	When it still won't talk	7
6.6	Devices without a network interface (qbridge)	7
6.7	Finishing up	9
7	Commands	9
7.1	Write methods: none, ramp, verify	9
7.2	Query LEDs: green, red, grey	10
8	The task grid	11
9	Nesting: sweeps inside sweeps	11
10	Linking: rows that track a driver	11
11	Nesting and linking combined	12
12	What combines with what	13
13	Virtual while-loops	13
14	Running, simulating, saving	14

1 License

qmeas is released under the **MIT License**. In plain terms: you may use, copy, modify, and redistribute it freely, including in commercial and closed-source work, provided the copyright notice and the license text below are kept with any substantial copy. There is no warranty — see the liability section that follows, which is part of the license, not an add-on.

MIT License

Copyright (c) 2026 Project h_e2

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

2 Liability and safety disclaimer

The warranty disclaimer above is legal boilerplate; this section spells out what it means for a program that drives real cryogenic and high-field hardware.

Watch out

qmeas sends commands to laboratory instruments — magnets, sources, cryostats — that can damage equipment, samples, or people if operated incorrectly. The software is provided without warranty of any kind. The authors accept **no liability** for any loss, damage, injury, or cost arising from its use, including from software defects, incorrect device configuration, mis-entered command strings, network faults, or instrument-specific behaviour the software cannot anticipate.

Practical consequences, all of which are your responsibility, not the software’s:

- **You are responsible for every command string you define.** qmeas transmits what you tell it to. A wrong SCPI string, a wrong ramp rate, or a wrong verify condition can drive an instrument outside safe limits. Verify command strings against the instrument’s own manual.
- **Hardware interlocks are not qmeas’s job.** Magnet quench protection, temperature limits, current limits, and emergency stops must be enforced by the instruments and their own safety systems. Do not rely on qmeas as a safety layer.
- **Test new sequences with Simulate, and at low/safe setpoints first.** Simulate shows what a task list will do without touching hardware. Use it before any long or high-stakes run. It is a convenience, not a guarantee.

- **Supervise runs that move real fields or temperatures.** The onhold and Stop mechanisms are best-effort; a network drop or a wedged instrument can prevent a clean abort.

By installing and using qmeas you accept these terms.

3 Installation

qmeas runs on **Windows** (the lab’s target platform) and needs Python plus a few packages. The optional AI Assistant additionally needs Ollama; that part can be skipped entirely — the measurement software is fully functional without it.

3.1 Step 1 — Python

Install **Python 3.11 or newer** (64-bit) from python.org. The python.org installer also installs the **Windows Python launcher** `py`, which every command in this manual uses — it finds the right Python via the registry, so it works even when `python` on `PATH` points at another installation (Anaconda, an old version) or at nothing. (Ticking “Add Python to `PATH`” during install is still fine, just no longer load-bearing.)

Confirm it in a *new* PowerShell or Command Prompt window:

```
py -version
```

3.2 Step 2 — Python packages

qmeas depends on three third-party packages; everything else it uses is in Python’s standard library. Install them with `pip` *through the launcher*:

```
py -m pip install wxpython matplotlib pyvisa pyvisa-py
```

Always use `py -m pip`, not bare `pip`, and launch qmeas with `py`: the two then use the *same* Python. On machines with several Pythons (Anaconda, old installs), bare `pip` and `py` can point at different interpreters — the classic symptom is `ModuleNotFoundError: No module named 'wx'` at startup even though the install “worked”.

What each is for:

<code>wxpython</code>	The GUI toolkit — all windows, panels, the task grid. Required.
<code>matplotlib</code>	The live Graph window. If it’s missing, qmeas still runs and measures; only the plot window shows a placeholder.
<code>pyvisa</code>	The instrument-communication layer (GPIO, USB, serial, TCPIP). Required for talking to any real device.
<code>pyvisa-py</code>	Pure-Python VISA backend, used automatically as a fallback when no vendor VISA library (NI-VISA) is present. Covers TCPIP/socket instruments — which is most of this lab.

Watch out

GPIO or USB instruments will not work until you additionally install the NI-VISA library ([download from ni.com](https://www.ni.com)). The pure-Python `pyvisa-py` backend only handles TCPIP/socket devices but not GPIO or USB! If qmeas can’t load a vendor library, it falls back to `pyvisa-py` automatically and warns you once.

3.3 Step 3 — Run qmeas

Put the qmeas files (qmeas.py and its companion modules, the images/, devices/, and custom/ folders) in one folder, open a terminal there, and run:

```
py qmeas.py
```

That's the whole measurement install. The next section is only needed for the optional AI Assistant.

4 Optional — the AI Assistant (Ollama)

qmeas includes a built-in AI assistant, opened via **View** → **AI Assistant**. It can answer questions about qmeas — what a red LED means, how nesting works, why a row failed — and help you set up an experiment: which device settings, commands, and task rows a measurement needs. It is optional; qmeas measures perfectly well without it.

The assistant runs a language model **locally** through Ollama — nothing is sent to any cloud service, which is the point: measurement data and device configurations never leave the machine.

Watch out

This is a **local model with limited capabilities** — far smaller than cloud AI services — and it **may give incorrect answers**, including confidently wrong ones. Treat its output as a starting point, not an authority: check suggested device settings and command strings against the instrument's own manual, and use Simulate before running anything it helped set up.

4.1 Step 1 — Install Ollama

Download the Windows installer from ollama.com/download/windows and run it (no administrator rights needed for a standard install). It installs a background service and a system-tray app, and exposes a local API at <http://localhost:11434>. If Windows Firewall prompts on first start, allow access for **private** networks — this is local only, nothing is exposed to the internet.

Confirm in a new terminal:

```
ollama -version
```

4.2 Step 2 — Pull a model

Models are downloaded with `ollama pull`. The recommended model is `llama3.1:8b` — it fits an 8 GB GPU and, in testing, follows the assistant's structured instructions (e.g. the two ways to set up a magnet's verify) noticeably more reliably than similarly-sized alternatives:

```
ollama pull llama3.1:8b
```

Test that it responds:

```
ollama run llama3.1:8b "Reply with the single word: ok"
```

If you have a larger GPU (or a remote GPU machine), a bigger model gives better answers on multi-step setup questions — see the table below. What matters most for this assistant is instruction-following, not raw size, so a strong 8B is a good default rather than a compromise.

4.3 Step 3 — Hardware and model size

Local models are limited by GPU memory (VRAM). Rough guide for quantised (Q4) models:

GPU VRAM	Comfortable model	Notes
8 GB	8B (llama3.1:8b)	Recommended; fits entirely in VRAM and answers fluently.
12 GB	13–14B	A step up if it fits fully in VRAM.
24 GB	32B	Good middle ground.
48 GB+	70–72B (e.g. llama3.1:70b)	Best answers on multi-step setup questions.

Key rule: pick a model that fits ENTIRELY in your GPU’s VRAM. A model that spills into system RAM/CPU (e.g. a 14B on an 8 GB card, or a 70B without the VRAM for it) becomes drastically slower — usually worse than a smaller model that fits. On the common 8 GB lab cards (e.g. NVIDIA T1000), llama3.1:8b is the right choice; a 14B is not worth it there.

No suitable GPU? Models still run on CPU, just slowly — fine for occasional questions, tedious for anything interactive. A large model on CPU is impractically slow, so use an 8B there.

Tip

Ollama unloads a model from memory after 5 minutes idle by default, so the next question waits for a reload. If the assistant feels sluggish between questions, set the environment variable `OLLAMA_KEEP_ALIVE` to 30m (or longer) to keep the model resident.

4.4 Step 4 — Use it

Open **View** → **AI Assistant**. qmeas connects to Ollama at its default local address (localhost port 11434) and lets you choose among the models you’ve pulled. No further configuration is needed beyond having Ollama running with at least one model downloaded.

5 The five panels

qmeas is a set of dockable panels. Drag any panel’s title bar to move or undock it; the **View** menu shows/hides panels and offers vertical or horizontal stacking presets.

- **Devices & Commands** — the tree of instruments and the read/write commands you’ve defined for each. Double-click a command to drop it into the task list.
- **Device Editor** — add or edit an instrument’s address and connection settings. Hidden until you click a device or open it from **View**.
- **Tasks** — the ordered grid of steps that make up a run. Most of this manual is about building rows here.
- **Log** — a timestamped record of each executed row.
- **Graph** — a *separate window* (not a docked panel, so you can throw it on a second monitor) that live-plots the running sweep.

Two virtual entries always sit in the device tree: `control` (pause, prompt, stop, counter, and while-loops — §13) and `script` (runs a Python file from the `custom/` folder; off by default).

6 Connecting a device

This is where most first-time trouble lives, so it gets the most space. Open the **Device Editor**, click **New Device**, and work through the buttons.

6.1 Scan first — but don't trust an empty result

Scan asks the VISA layer to list everything it can see: GPIB, USB, serial, and instruments that answer VISA's LAN discovery.

Watch out

A device that does *not* appear in the scan is very often still there and perfectly usable. Most LAN instruments in this lab (lock-ins, magnet controllers, source-measure units) speak over a plain **raw TCP socket** and have no way to announce themselves — the scan simply can't see them. **This is normal.** You add them by hand (next section). An empty or short scan list is not a fault and not a reason to stop.

6.2 Adding a socket (LAN) device by IP

Click **Add TCPIP...**, enter the instrument's IP address and port, and choose *Raw socket (SOCKET)* — the common case here. (Choose *Standard VISA (INSTR)* only if the instrument's manual explicitly documents VXI-11 or HiSLIP.)

You must know the IP before this step; qmeas cannot discover it for a socket device. The networking pitfalls, in rough order of how often they bite:

- **Same subnet, or nothing connects.** The instrument and the qmeas PC must be on the same local network — in practice the first three numbers of their IPs match, e.g. both 192.168.1.x. If they differ (192.168.1.x vs 192.168.2.x), the connection just times out. That's a network problem, not a qmeas problem, and no setting inside qmeas will fix it.
- **DHCP moves addresses.** Most instruments ship set to automatic (DHCP) addressing. Convenient — until the instrument is power-cycled and comes back on a *different* IP, silently breaking a device that worked yesterday. For anything permanent, either set a static IP in the instrument's own network menu, or reserve its address on the lab router (keyed to its MAC). Then it never moves again.
- **Find the current IP** from the instrument's front-panel network/LAN menu, or your router's client list.
- **Ping before qmeas.** From the qmeas PC, run `ping <ip>`. No reply $\Rightarrow$ fix the network first; qmeas will do no better than ping can.

Example — a magnet controller on a raw socket

```
IP      192.168.1.42
Port    7020
Type    Raw socket (SOCKET)
```

6.3 HTTP / JSON-RPC devices

Some instruments expose an HTTP server rather than a socket (e.g. the Kiutra cryostat's JSON-RPC API). Click **Add HTTP...** and give the host, port, path, and Content-Type. Each command is then sent as an HTTP POST with your command string as the body; the response body comes back as the reading. The command strings themselves (the JSON-RPC or REST text) are defined later in the Command Editor, exactly like SCPI strings.

Example — Kiutra JSON-RPC

Host	192.168.1.55
Port	1006
Path	/
Content-Type	application/json-rpc

6.4 Termination characters matter

Every instrument expects each message to end with specific character(s) — the **write/read termination**. Get these wrong and the symptom is maddening: the connection opens fine, but every command times out or returns nothing, because the instrument is still waiting for an end-of-message it never received. Most raw-socket SCPI gear here uses `\n` for both. When in doubt, the instrument’s programming manual states it explicitly — check it before assuming the device is broken.

6.5 When it still won’t talk

Tip

Before deeper debugging: **restart the instrument**. Single-client LAN instruments can get stuck holding a half-open connection from a previous crash or an ungraceful disconnect, and will refuse or drop new connections until power-cycled. A restart clears this surprisingly often — try it early, not as a last resort.

6.6 Devices without a network interface (qbridge)

Some instruments have *no* instrument interface at all — no SCPI, no socket, no VISA. The Quantum Design OptiCool is the standing example: it is controlled exclusively through its MultiVu software’s OLE interface on the control PC. qmeas talks to such devices through a **qbridge**: a small translator process, shipped with qmeas in the `qbridge/` folder, that speaks the vendor’s interface on one side and plain socket text on the other. To qmeas, the result is an ordinary raw-socket device.

OptiCool setup (once, on the OptiCool control PC):

1. MultiVu running, as always.
2. Install Quantum Design’s Python package and start its server next to MultiVu: `py -m pip install MultiPyVu` then `py -m MultiPyVu`.
3. Start the bridge: `py opticool.py` (from the `qbridge/` folder; listens on port 5100). Set both to auto-start with Windows so you never think about them again.

Then, in qmeas on any lab PC: **Add TCPIP...** with the control PC’s IP, port 5100, raw socket, terminators `\n` — exactly like any other LAN instrument (§6.2 applies, including Test Connection, which the bridge answers). Define commands from this verb table:

Command string	Meaning
TEMP?	temperature (K), e.g. 4.2130
TSTATE?	temperature status: Stable, Tracking, Near, Chasing, Standby
AUXTEMP?	auxiliary thermometer (K)
FIELD?	field (Oe; 10000 Oe = 1 T)
FSTATE?	field status: Holding (driven), Ramping, ...
TEMP [%],10	set temperature to the row value, 10 K/min
FIELD [%],110	set field to the row value (Oe), 110 Oe/s
FHOLD	freeze the magnet at its current field — use as this device's onhold command so Stop/abort parks the magnet

Everything then works as for any device: sweep `FIELD [%],110` rows, log `TEMP?` green during any sweep, *verify* a field step on `FSTATE?` equals `Holding (driven)` (type the state string exactly), or gate with a while-row on `TEMP?`. Waiting for stability is deliberately done in qmeas — visible, abortable, logged — not inside the bridge.

Tip

If OptiCool reads fail (`READ FAILED` in the data), the usual suspects in order: the bridge process isn't running; the MultiPyVu server isn't running; MultiVu isn't running. Restart in that order — the bridge retries its upstream connection once by itself, so a brief MultiPyVu hiccup usually heals without intervention.

Note

Two OptiCool adapter variants ship, serving identical commands: `opticoool.py` (via Quantum Design's MultiPyVu — preferred, but needs current OptiCool software) and `opticoool_qdi.py` (via the QDInstrument .NET server on port 11000, the LabVIEW interface — works on older software, can run on the qmeas PC itself, needs `py -m pip install pythonnet` and `QDInstrument.dll` beside it). Only the bridge changes; qmeas configuration is the same for both. State strings differ between the two — always `Query Once` before using one as a verify target.

A third shipped adapter, `toptica_bridge.py`, covers TOPTICA DLC pro/DLC smart controllers (TeraScan): their command line is session-oriented (telnet banner plus an access-level handshake), so the bridge holds one SDK session and exposes the whole DeCoF parameter tree generically — qmeas command strings `GET <param>`, `SET <param> [%]`, `EXEC <cmd>` reach any parameter without adapter changes (`py -m pip install toptica_lasersdk`; run `py toptica_bridge.py -toptica-host <ip>`). Verified TeraScan (DLC smart THz) commands, units GHz:

GET frequency:frequency-act	actual THz frequency — log it green: the readback honestly wanders at the MHz level (the THz output is a two-DFB-laser beat), so recording the actual beats trusting the setpoint
SET frequency:frequency-set [%]	frequency setpoint; use a FLOAT number format (DeCoF real-typed parameters reject bare integers)
GET frequency:fast-scan-isscanning	1 while the native fast scan runs, else 0

Point-by-point stepping (a `SET ...frequency-set [%]` sweep row with the reads green) is the

qmeas-idiomatic mode and composes with nesting; the native continuous TeraScan sweep does not map onto task rows. Discover further parameter names with the `decof_tree_dump.py` helper or the TeraScan software’s parameter reference. Note that each qmeas read opens a short-lived connection *to the bridge* (visible in its log) — normal; the bridge’s own session to the controller persists.

Other interface-less instruments follow the same pattern: copy an adapter file in `qbridge/`, map a handful of verbs to the vendor’s API, and the device becomes a socket instrument.

6.7 Finishing up

Fill in the **Alias** (lowercase letters/digits, e.g. `magnet`; every command becomes `alias_commandname`), termination, timeout (milliseconds; 5000 is fine for almost everything), and the device list it joins. Click **Test Connection** — for SCPI it sends `*IDN?` and shows the reply; for HTTP it checks the host/port are reachable. Then **Save Device**.

7 Commands

A new device has no commands. Right-click it → **Add Command...** A **write** sends a value; `[%]` in its command string is where qmeas substitutes the value from the task row. A **query** reads a value back; use **Query Once** to see the raw reply and click-select the part you want.

Example — two commands on a source aliased `gate1`

Write:	name <code>gate1_setvoltage</code>	string <code>VOLT [%]</code>
Query:	name <code>gate1_readcurr</code>	string <code>CURR?</code>

7.1 Write methods: **none**, **ramp**, **verify**

A write command can carry a **Method**, chosen from a dropdown in the Command Editor (visible only for a write command that contains a `[%]` placeholder). It governs *how* the value is delivered.

none — write the value in one shot and move on. The default; correct for almost everything.

ramp — approach the target gradually instead of jumping. qmeas itself steps the value in one-second increments, each no larger than the rate you set, all the way from the current value to the target. Use it whenever a sudden jump would be unsafe or unphysical (a gate that must not slew, say). Two fields appear:

- **Ramp rate (units/s)** — the maximum change per second, in the same units as the command’s value. Must be greater than zero.
- **Based on output state of** — a read-back (query) command on the *same* device, used to find the *starting* value so qmeas knows where the ramp begins. Pick the query that reports this instrument’s present output.

Watch out

Ramp is **write-and-forget**: qmeas issues the intermediate setpoints on a one-second clock but does *not* check the instrument actually reached each one. It paces commands; it does not confirm the hardware kept up. If the instrument has its own internal ramp/slew feature, decide which one owns the rate — don’t set both against each other. The ramp rate is your responsibility to keep within the instrument’s safe limits.

verify — after writing, repeatedly read a chosen read-back until it reports the expected value, or give up. This is how a step waits for hardware: a field setpoint that blocks until the magnet

controller reports HOLD, for instance. Fields:

- **Based on output state of** — the read-back (query) command to poll, on the same device.
- **...equals** — the value that read-back must report for the step to be considered done. For a status string, type it exactly (HOLD); for a number, the target value.
- **Tolerance ($\pm$, 0 = exact)** — how close a numeric read-back must be to count as a match. Leave at 0 for an exact match (the right choice for status strings like HOLD); widen it for a numeric setpoint that will never land on the exact digits (e.g. a field good to ± 0.0005 T).
- **Verify timeout (s, 0 = none)** — how long to keep polling before giving up. On timeout the run **aborts** (with magnet onhold); 0 means wait indefinitely.

Example — a field setpoint that waits for HOLD

```
Command: magnet_setfield      string SOUR:FIELD [%]
Method:  verify
Based on output state of: magnet_status
...equals: HOLD      Tolerance: 0      Verify timeout: 600
```

Put that command in a task row with the field value in Constant/Start, and the row won't finish — and the next row won't begin — until `magnet_status` reads HOLD or 600 s elapse.

Tip

Ramp vs. verify are independent, not two halves of one thing. Ramp paces the *outgoing* setpoints; verify waits on the *incoming* read-back. A magnet controller with its own internal ramp is the classic verify case: you write the setpoint, let the controller ramp on its own, and *verify* that it reports HOLD when done — no qmeas ramp involved. A source with no internal ramp is the classic ramp case. You pick one method per command.

7.2 Query LEDs: green, red, grey

Every query command shows a small coloured LED next to it in the Devices & Commands tree. Click it to cycle through three states, which control whether that reading is taken during a run and whether it's saved:

- **Green — read and save.** The query is polled at every measurement step and its value is written to the data file. This is the normal state for any reading you want to record (a lock-in output, a field value).
- **Red — read for verification, don't save.** The query is still available for a write command's *verify* method (§7.1) to poll, but it is *not* added to the data file as a measured column. Use it for a status read-back — e.g. `magnet_status` watched by a verify step — that you don't want cluttering the saved data with a HOLD column.
- **Grey — off.** The query is not polled at all and not saved.

Red and grey behave identically as far as the data file is concerned (neither is saved); the difference is a reminder to you that a red query is deliberately in use *somewhere* as a verify target, whereas grey is simply idle. The state is remembered when you close qmeas and restored next time.

Tip

If a status string like HOLD or a READ FAILED message is showing up as a column in your data files and you don't want it there, the query is green — click its LED to red. It stays

available to any verify step that uses it, but drops out of the saved data.

8 The task grid

Each row is one step, run top to bottom. Columns:

On	Checkbox. Unchecked rows are skipped, not deleted.
Command	device_command, dragged from the tree.
Constant/Start	The single value (constant write) or the sweep's first value. On a linked row, a [%] expression instead (§10).
Final	The sweep's last value. Blank = constant write.
Steps	Number of points, Start→Final. Blank/0 = constant.
Integration Time	Wait after each step: 2s, 0.5s, 2m (bare number = seconds; ms not accepted — use 0.5 not 500ms).
Comment	Free text, copied into the Log.

Tip

Fields turn **red** when a row is inconsistent — e.g. you filled Final but left Steps or Integration Time empty, or typed a non-number. A red cell never runs the way you'd want; fix it before pressing Start. (This is why the earlier 10a-in-Steps class of silent mistake can't slip through unseen.)

The control device gives four fixed rows: **pause** (waits the Constant value in seconds), **userprompt** (popup, waits for you), **stop** (aborts, same as the Stop button), and **counter** (a step/time index that writes no instrument — behaves like a sweep for nesting and linking).

9 Nesting: sweeps inside sweeps

Nesting chains rows into nested for-loops: for each step of the outer row, the inner row runs its whole sweep. Select the row that should be inner, right-click → **Nest**; it attaches to the row directly above, one level deeper. Build deeper chains bottom-up, one Nest at a time. Max depth 4. **Unnest** peels from the bottom.

Example — gate sweep at every field

```
Row 1 magnet_setfield      Start 0 Final 1 Steps 6 IntTime 5s
Row 2 gate1_setvoltage Start -2 Final 2 Steps 21 IntTime 0.2s (nested)
```

Right-click Row 2 → Nest. The run sets field to 0 T, sweeps the gate through all 21 points, steps the field to 0.2 T, sweeps the gate again, ... $6 \times 21 = 126$ points. A counter can be the outer loop too — “every 10 s, sweep the gate” for a drift study:

Example — counter as the outer (time) loop

```
Row 1 control_counter      Steps 100 IntTime 10s
Row 2 gate1_setvoltage Start -2 Final 2 Steps 21 IntTime 0.2s (nested)
```

10 Linking: rows that track a driver

Linking makes “follower” rows compute their value from a “mother” row's swept value, instead of sweeping on their own. Select the follower, right-click → **Link**; it attaches to the row above. Its Constant/Start holds an **expression** with [%] standing for the mother's current value. Operators + − ∗ / ^ and functions sin cos tan exp log log10 sqrt abs (and pi, e) are available.

Example — symmetric split gates (gate2 = -gate1)

Row 1	gate1_setvoltage	Start -2 Final 2 Steps 41 IntTime 0.3s	(mother)
Row 2	gate2_setvoltage	Start [%]*-1	(follower)

At the step where gate1 = +0.5 V, gate2 is written to -0.5 V before the reading. A follower can itself use a ramp/verify method and a nonlinear expression:

Example — heater tracking a time index nonlinearly

Row 1	control_counter	Steps 50 IntTime 1s	(mother)
Row 2	heater_setpower	Start 0.5*sin([%]/8)^2	(follower)

Tip

A follower's Final, Steps, and Integration Time are driven by the mother, so they're locked. Its Constant/Start (the expression) is *required* — an empty one turns red.

Watch out

[%] is substituted into a Python expression, so a number glued directly to it is a syntax error: write [%]*1e3, not [%]e3 or [%]1e3. Such expressions are rejected with a hint before anything is written to hardware.

11 Nesting and linking combined

Since version 2.1 a link box may sit at *any* nesting level: every row of a nested chain can be a mother with up to three followers. The followers sit directly under their mother, at the *same* depth, and the next deeper row (the mother's nested child) follows *after* the box. Followers are leaves — nothing can be nested under a follower.

Example — two generators tracking a counter, inside a 2D map

Row 1	magnet_setfield	Start 1 Final 10 Steps 9 IntTime 30s	
Row 2	smu_setvoltage	Start 1 Final 2 Steps 10 IntTime 0.2s	(nested)
Row 3	control_counter	Start 1 Final 2 Steps 1 IntTime 1s	(nested)
Row 4	freq1_setfreq	Start [%]*1e3	(follower of 3)
Row 5	freq2_setfreq	Start [%]*1e3+100	(follower of 3)

For each field, for each SMU voltage, the counter steps through its values; after each counter step both generators are written, then the counter's Integration Time runs, then the readings are taken. The box can also sit on a *middle* level, with a deeper sweep after it:

Example — followers on the middle level, a deeper sweep below

Row 1	a_set	Start 0 Final 1 Steps 10	
Row 2	b_set	Start 0 Final 1 Steps 10	(nested)
Row 3	c_set	Start [%]*2	(follower of Row 2)
Row 4	k_set	Start 0 Final 1 Steps 10	(nested under Row 2)

At every step of b, c is written to 2b before the whole k sweep runs.

Building it. Two equivalent routes for the follower: add it directly under its mother at the same depth and Link it (a sibling), or Nest it under the mother first and then Link it — Link pulls it up to the mother's depth. The nested child is added *below the box* and nested as usual (it attaches under the box's mother). Box members can't be nested or unnested while linked — Unlink first.

Data file. After each level's count/value pair come that level's follower columns <alias>_value,

e.g. for the first example: `outer_count outer_value inner1_count inner1_value inner2_count inner2_value freq1_setfreq_value freq2_setfreq_value elapsed_s ....` Files from chains without links are unchanged.

Limits. Max depth 4, max 4 rows per box (mother + 3), one box per level. Every follower of every level is validated before the first write; a bad expression anywhere aborts the whole chain before it starts.

12 What combines with what

Mother rows and nesting parents must have a value to sweep; the value-less row types (while, script, pause/prompt/stop) can only sit as plain standalone rows. A row is either a nesting level (possibly a link mother) or a follower, never both; followers have no children (§11). Everything below is enforced both in the menus (grayed out) and at run time.

Row type	Nest parent	Nest child	Max depth	Link mother	Link follower
Device write (Steps ≥ 1)	✓	✓	4	✓	✓
<code>control_counter</code>	✓	✓	4	✓	×
Virtual while-loop	×	×	—	×	×
<code>script_*</code>	×	×	—	×	×
<code>pause/userprompt/stop</code>	×	×	—	×	×

The value-less rows compose by *sequence*, not structure — just put them one after another as ordinary rows:

Example — gate a nested sweep behind a hardware wait

```

Row 1 magnet_setfield      Start 1 (method verify: waits for readback = 1 T)
Row 2 control_waitforhold (while: magnet_status = HOLD, timeout 600s)
Row 3 control_counter      Steps 100 IntTime 10s
Row 4 gate1_setvoltage Start -2 Final 2 Steps 21 IntTime 0.2s (nested)
Row 5 script_saveplot

```

13 Virtual while-loops

A while-loop is a `control` command that repeats until a condition on some reading is met, then continues — like `verify`, but as its own row that also *records data every poll*. Right-click `control` → **Add Virtual While...**, pick a query to watch (from any device), an operator (`=` `!=` `<` `>`), a value, and a timeout (mandatory; 0 = wait forever). Drop it in the grid; its Integration Time becomes the poll interval (minimum 0.1 s).

Example — wait for base temperature, logging as it goes

```

Definition: watch cryostat_temperature, operator <, value 0.05, timeout 0
Grid row:   control_waitforbase      IntTime 5s

```

Every 5 s it reads the temperature (and all other active queries, saved to the data file) and exits once it drops below 50 mK.

Watch out

Any while-loop failure — timeout reached, a non-numeric reading against `</>`, the watched device switched off — **aborts the whole run** (with magnet onhold), by design: the row exists to gate progress on real hardware state, so if it fails, later rows would run under

unknown conditions.

14 Running, simulating, saving

Start runs the list; **Stop** aborts (holding magnet devices). **Simulate** walks the list and prints, in plain language, what each row would do — sweep ranges, nesting/linking with computed follower values, while conditions — *without touching hardware*. It applies the same rules as a real run, so it also flags invalid combinations. Use it to sanity -check a list before a long run; it's optional, not required. **Save / Load** store and restore a task list. Each sweep writes its own data file, which the Graph window plots live.